

Benchmarking 3DVar with Different Optimization Methods

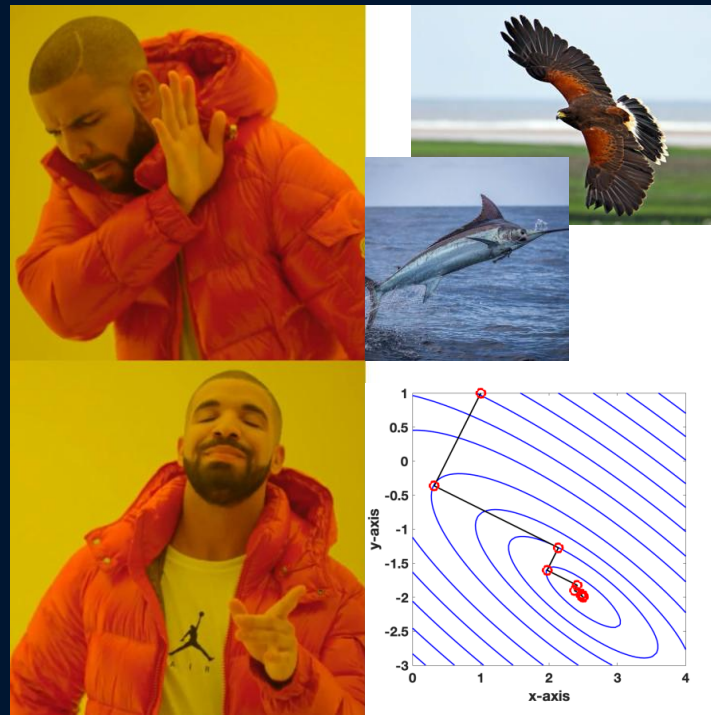
b12209017 顏子傑

Some Changes of My Project

What happened to my midterm plan

Swarm-based algorithms do not have an advantage when an analytical Jacobian is available.

Turned to comparing different gradient-based optimization algorithms.



OSSE Configuration

Model and DA Method

Model: Lorenz 96 ($F = 8.0$)

DA method: 3D-Var

other details:

```
1  dt = 0.05 / 6      # Integration time step (1hr)
2  assim_window = 6    # Steps between analysis (6hrs)

1  # >>> nature run >>>
2  x_init = F * np.ones(N)
3  x_init[19] += 0.01 # Initial perturbation
4  # model spin-up
5  x_spinup = model_driver.integrate(x_init, steps=int(100*t_per_day/dt)) # 100 days of spin-up
6  x_true_0 = x_spinup[-1]
7  # Generate the truth
8  x_truth = model_driver.integrate(x_true_0, steps=total_steps)
9  # <<< nature run <<<
```

OSSE Configuration

Optimization Algorithms

```
1 def gradient_descent_optimizer(func, jac, x0, args=(), tol=tol, maxiter=maxiter):
2     """Gradient descent (GD)"""
3     x = np.array(x0, dtype=float)
4
5     for t in range(maxiter):
6         g = jac(x, *args)
7         e = -g
8
9         if np.linalg.norm(g) < tol: break
10
11         alpha = line_search(func, jac, x, e, g, args=args)[0]
12         if alpha is None: alpha = 1e-4
13
14         x = x + alpha * e
15
16     print(f"GD optimizer converged in {t+1} iterations.")
17     return x
```

```
1 def conjugate_gradient_optimizer(func, jac, x0, args=(), tol=tol, maxiter=maxiter):
2     """Conjugate Gradient method with robust restarts"""
3     x = np.array(x0, dtype=float)
4
5     g = jac(x, *args)
6     e = -g
7
8     for t in range(maxiter):
9         if np.linalg.norm(g) < tol: break
10
11         alpha = line_search(func, jac, x, e, g, args=args)[0]
12         if alpha is None: alpha = 1e-4
13
14         x_new = x + alpha * e
15         g_new = jac(x_new, *args)
16
17         # Polak-Ribiere
18         beta_PR = max(0, np.dot(g_new, g_new - g) / np.dot(g, g))
19
20         e = -g_new + beta_PR * e
21
22         x = x_new
23         g = g_new
24
25     print(f"CG optimizer converged in {t+1} iterations.")
26     return x
```


OSSE Configuration

Optimization Algorithms

1. Momentum (m)

- Accumulates past gradients
- Reduces zig-zag oscillations
- Accelerates convergence along valleys

2. Adaptive Learning Rate (v)

- Adjusts step size for each parameter
- Smaller steps in steep directions
- Larger steps in flat directions

```
1 def adam_optimizer(func, jac, x0, args=(), lr=0.01, beta1=0.9, beta2=0.999,
2                     eps=1e-8, maxiter=maxiter):
3     """Adam optimization algorithm."""
4     x = np.array(x0, dtype=float)
5     m = np.zeros_like(x)
6     v = np.zeros_like(x)
7     x_best = x.copy()
8
9     for t in range(1, maxiter + 1):
10         g = jac(x, *args)
11
12         m = beta1 * m + (1 - beta1) * g
13         v = beta2 * v + (1 - beta2) * (g ** 2)
14
15         m_hat = m / (1 - beta1 ** t)
16         v_hat = v / (1 - beta2 ** t)
17
18         x -= lr * m_hat / (np.sqrt(v_hat) + eps)
19         lr *= 0.95 # Decay Learning rate
20
21         if func(x, *args) < func(x_best, *args):
22             x_best = x.copy()
23
24         if np.linalg.norm(g) < tol:
25             break
26
27     print(f"Adam optimizer converged in {t} iterations.")
28     return x_best
```

Gradient:

$$g_t = \nabla J(x_{t-1})$$

First Moment (Momentum):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

Second Moment (Adaptive Learning Rate):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Bias Correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Parameter Update:

$$x_t = x_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t$$

OSSE Configuration

Exp. Under Varying Complexity

Increasing complexity of cost function



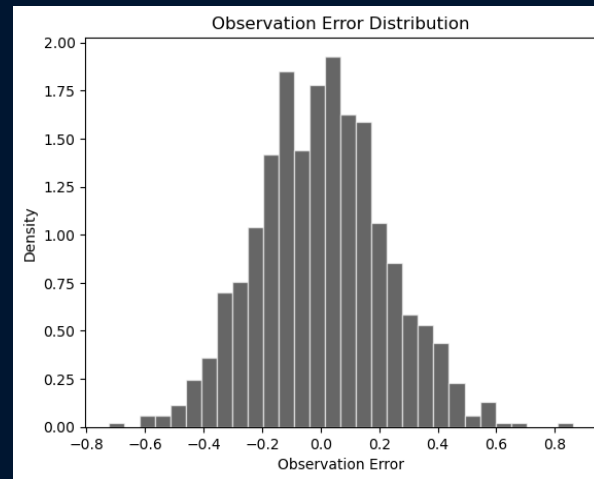
	Exp.I	Exp.II	Exp.III
H	all grids	all grids	$\frac{1}{2}$ grids
R	diagonal	diagonal	diagonal
B	diagonal	1 grid length covar.	1 grid length covar.

OSSE Configuration

Simulating observation

Obs. every other grid point.

```
1  # >>> simulating observations >>>
2  obs_variance = 0.1
3  obs_count = N // 2
4  H = np.zeros((obs_count, N))
5  for i in range(obs_count):
6      H[i, 2*i] = 1.0
7  R = obs_variance * np.eye(obs_count)
8
9  observations = []
10 for cycle in range(1, total_cycles + 1):
11     t_idx = cycle * assim_window
12     truth_state = x_truth[t_idx]
13     obs = H @ truth_state \
14         + np.random.normal(0, np.sqrt(obs_variance), obs_count)
15     observations.append(obs)
16 # <<< simulating observations <<<
```

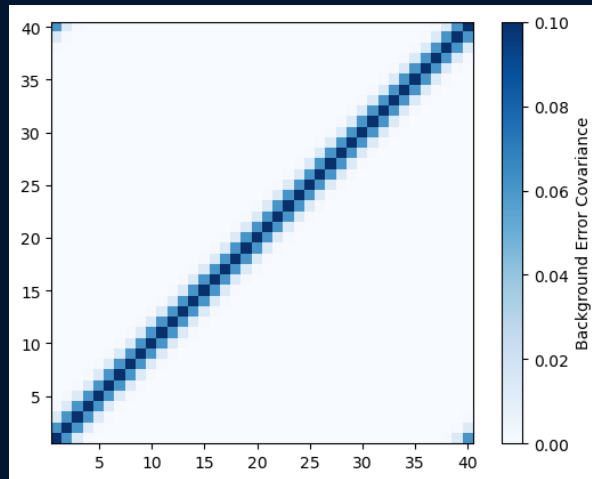


OSSE Configuration

Background Error Covariance

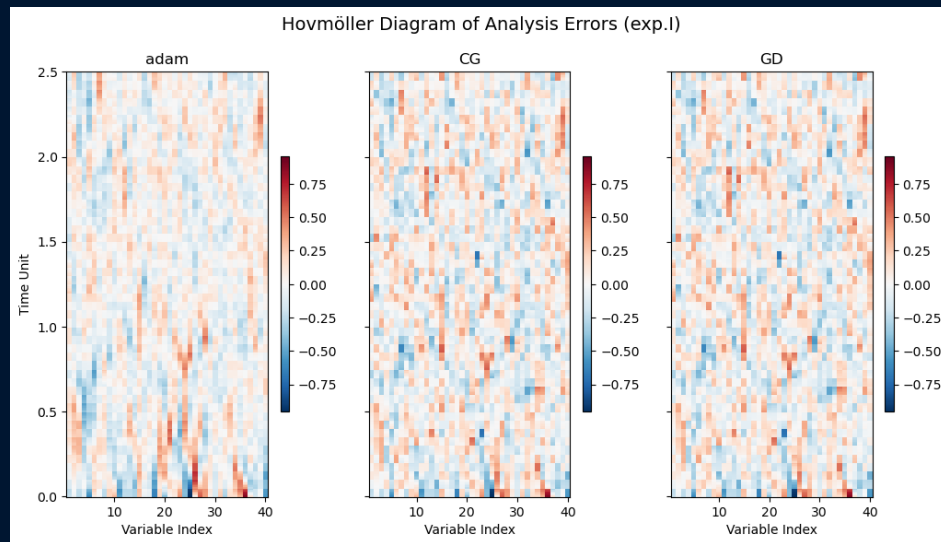
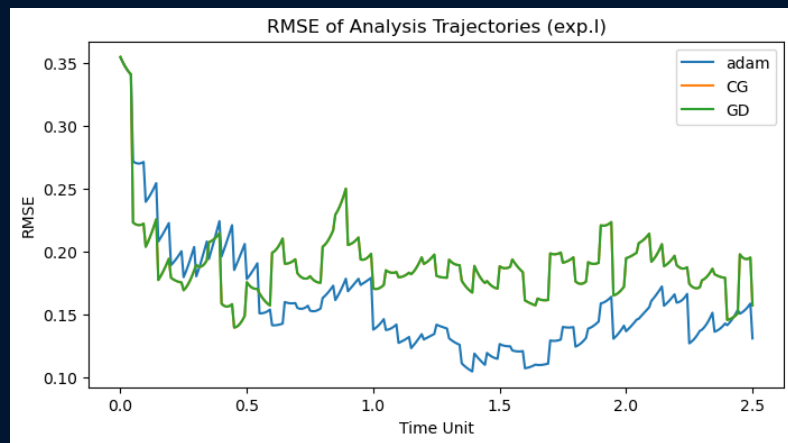
Covariance from nearby grids

```
1 # >>> background error covariance >>>
2 B_variance = 0.1
3 B = np.zeros((N, N))
4 for i in range(N):
5     for j in range(N):
6         dist = min(abs(i - j), N - abs(i - j))
7         B[i, j] = B_variance * np.exp(-(dist**2) / 2)
8 # <<< background error covariance <<<
```



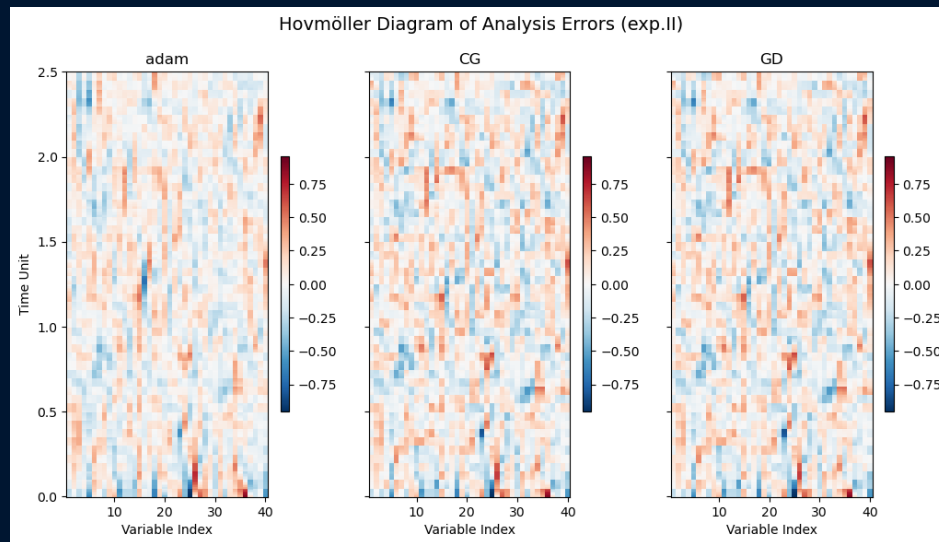
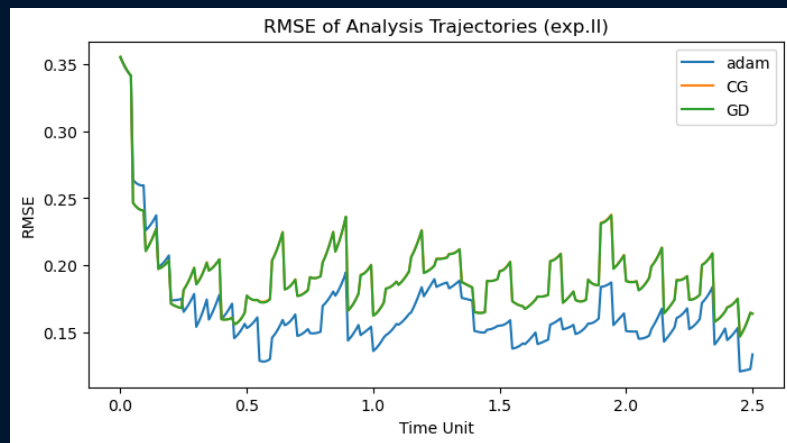
Results - Exp.I

- CG and GD converge to identical analysis states in the same number of iterations.
- Adam slightly outperforms.



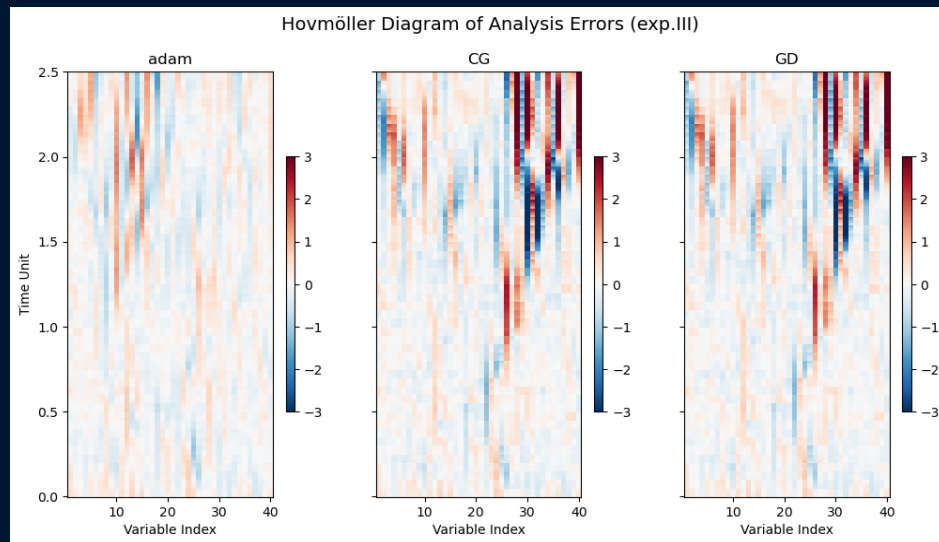
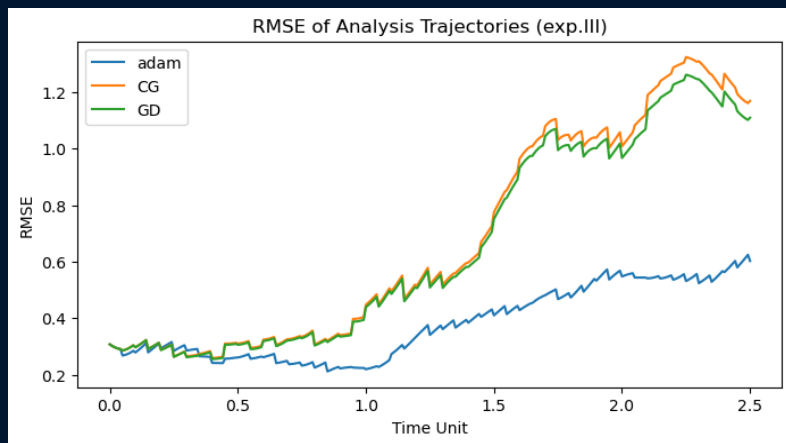
Results - Exp.II

- CG converges in half the iterations with nearly identical analysis states as GD.



Results - Exp.III

- Under the same max-iteration limit, GD performs better than CG.
- Adam achieves markedly better performance.



THANKS!

src: github.com/JackQuark/DA2026_FinalProject